VOLUME TOKEN ADMIN GUIDE

CRITICAL CONTRACT ADDRESSES

- **Token Contract:** `0x5D3a740cb69Df9e36389C9DD2e2015F8408A3f52` (Sepolia)
- **Uniswap Pair:** `0x0b9FEeF5685c6883ae655A200e3AF479aB665916` (Sepolia)
- **Owner Address:** `0x935B3aC12B0CE29dd28cbCefCee8121354233e49`

ADMIN FUNCTIONS REFERENCE

🛡️ SECURITY CONTROLS

Emergency Pause System

```javascript
// PAUSE TRADING (Emergency only)
await token.pause();

// RESUME TRADING (After emergency resolved)
await token.unpause();

// CHECK PAUSE STATUS
const isPaused = await token.paused();
console.log("Contract paused:", isPaused);
```

Blacklist Management

```javascript
// ADD ADDRESS TO BLACKLIST
await token.setBlacklisted("0xAddressToBlock", true);

// REMOVE FROM BLACKLIST
await token.setBlacklisted("0xAddressToUnblock", false);

// CHECK BLACKLIST STATUS
const isBlacklisted = await token.blacklisted("0xAddressToCheck");
```

⚙️ PROTECTION SETTINGS

Anti-Whale Controls

```javascript
// SET MAX TRANSACTION AMOUNT (current: 0.5% = 250M tokens)
const newMaxTx = ethers.utils.parseEther("250000000"); // 250M tokens
await token.setMaxTxAmount(newMaxTx);

// SET MAX WALLET AMOUNT (current: 1% = 500M tokens)
const newMaxWallet = ethers.utils.parseEther("500000000"); // 500M tokens
await token.setMaxWalletAmount(newMaxWallet);

// ENABLE/DISABLE ANTI-WHALE
await token.setAntiWhaleEnabled(true); // or false
```

Fee Management

```javascript
// SET BUY FEE (current: 2% = 200 basis points, max: 10%)
await token.setBuyFee(200); // 2%

// SET SELL FEE (current: 4% = 400 basis points, max: 10%)
await token.setSellFee(400); // 4%

// ENABLE/DISABLE FEES
await token.setFeesEnabled(true); // or false

// SET FEE DISTRIBUTION (LP vs Marketing)
await token.setFeeShares(40, 60); // 40% LP, 60% Marketing (must total 100%)
```

🎯 EXEMPTION MANAGEMENT

Fee Exemptions

```javascript
// EXEMPT ADDRESS FROM FEES
await token.setFeeExempt("0xAddressToExempt", true);

// REMOVE FEE EXEMPTION
await token.setFeeExempt("0xAddressToTax", false);
```

Transaction Limit Exemptions

```javascript
// EXEMPT FROM MAX TX LIMITS
await token.setMaxTxExempt("0xLargeHolder", true);

// EXEMPT FROM MAX WALLET LIMITS
await token.setMaxWalletExempt("0xExchange", true);

// EXEMPT FROM ANTI-DUMP (5 min hold requirement)
await token.setAntiDumpExempt("0xRouter", true);
```

🔧 UNISWAP INTEGRATION

Pair Management

```javascript
// SET UNISWAP PAIR ADDRESS
await token.setUniswapV2Pair("0xPairAddress");

// GET CURRENT PAIR
const currentPair = await token.uniswapV2Pair();
```

Router Setup for New Network

```javascript
// For mainnet deployment, update router address:
const MAINNET_ROUTER = "0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D";
// Note: Router address is set in constructor, cannot be changed after deployment
```

📊 MONITORING & QUERIES

Check Protection Settings

```javascript
```

```
// CURRENT LIMITS
console.log("Max TX:", ethers.utils.formatEther(await token.maxTxAmount()));
console.log("Max Wallet:", ethers.utils.formatEther(await token.maxWalletAmount()));
console.log("Anti-whale enabled:", await token.antiWhaleEnabled());

// CURRENT FEES
console.log("Buy fee:", await token.buyFee(), "basis points");
console.log("Sell fee:", await token.sellFee(), "basis points");
console.log("Fees enabled:", await token.feesEnabled());
```

Check Exemption Status

```javascript
// CHECK IF ADDRESS IS EXEMPT
const address = "0xAddressToCheck";
console.log("Fee exempt:", await token.feeExempt(address));
console.log("Max TX exempt:", await token.maxTxExempt(address));
console.log("Max wallet exempt:", await token.maxWalletExempt(address));
console.log("Anti-dump exempt:", await token.antiDumpExempt(address));
```

Anti-Dump Status

```javascript
// CHECK IF ADDRESS CAN SELL (after 5 min hold)
const canSell = await token.canSell("0xHolderAddress");
console.log("Can sell immediately:", canSell);
```

🚨 EMERGENCY PROCEDURES

Immediate Response Actions

1. **Suspicious Activity Detected:**

   ```javascript
   await token.pause(); // Stop all trading
   // Investigate issue
   // Apply blacklist if needed
   await token.setBlacklisted("0xMaliciousAddress", true);
   await token.unpause(); // Resume when safe
   ```

2. **Smart Contract Bug:**

```javascript
await token.pause(); // Immediate halt
// Deploy fixed version if needed
// Coordinate with community
```

3. **Whale Attack:**

```javascript
// Reduce limits temporarily
await token.setMaxTxAmount(ethers.utils.parseEther("100000000")); // Lower limit
// Monitor and adjust as needed
```

💰 TREASURY MANAGEMENT

Emergency Withdrawal

```javascript
// WITHDRAW ETH FROM CONTRACT (emergency only)
await token.emergencyWithdraw();
// This sends all ETH in contract to owner address
```

Fee Collection Monitoring

- Monitor fee accumulation in contract
- Track marketing fund distribution
- Verify liquidity additions

🔐 SECURITY BEST PRACTICES

1. Multi-Signature Recommendation

- Transfer ownership to multisig wallet for mainnet
- Require multiple signatures for critical functions
- Use Gnosis Safe or similar solution

2. Access Control

- Keep private keys secure and backed up
- Use hardware wallets for mainnet operations
- Document all admin actions

3. Community Communication

- Announce any parameter changes in advance
- Explain reasoning for emergency actions
- Maintain transparency with holders

📈 RECOMMENDED SETTINGS FOR MAINNET

Conservative Launch Settings

```javascript
// Anti-whale (can be relaxed later)
maxTxAmount: 0.5% of supply (250M tokens)
maxWalletAmount: 1% of supply (500M tokens)

// Fees (proven sustainable)
buyFee: 2% (200 basis points)
sellFee: 4% (400 basis points)

// Protection enabled
antiWhaleEnabled: true
feesEnabled: true
```

Addresses to Exempt Immediately

```javascript
// Essential exemptions for mainnet:
- Uniswap V2 Router: 0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D
- Uniswap V2 Factory: 0x5C69bEe701ef814a2B6a3EDD4B1652CB9cc5aA6f
- WETH: 0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2
- Your marketing wallet
- Your treasury wallet
- The token contract itself
```

🎯 REGULAR MAINTENANCE

Weekly Tasks

- Monitor holder distribution
- Check fee collection

- Review any reported issues
- Update community on metrics

Monthly Tasks

- Review and potentially adjust limits
- Analyze trading patterns
- Consider fee optimization
- Plan feature updates

⚠️ **CRITICAL REMINDER:** Always test changes on testnet first! Never make direct changes to mainnet without thorough testing.